Informe de Actualización Frontend

Módulo de Entradas — Grupo 3

El presente informe detalla las correcciones y mejoras implementadas en el repositorio general del frontend (`repoAyu/FrontendTitec-main` , sincronizado también en `repo/frontend`) por parte del equipo de Entradas. Estas modificaciones resuelven problemas críticos de integración y aseguran el correcto funcionamiento del flujo de emisión de tickets con el backend de microservicios.

Problemas Principales Resueltos:

- **Vista estática:** La ruta `/entradas` presentaba un componente de demostración estático. Se reemplazó por el selector interactivo real de eventos.
- **Falsos positivos de Tickets:** El flujo de reserva/emisión generaba un ID de ticket falso directamente en el navegador. Ahora, se consume el API real obteniendo el ticket emitido por el backend en MongoDB, el cual se sincroniza automáticamente con el módulo de Check-in.

Detalle de Archivos Modificados

Archivo en FrontendTitec-main	Qué se modificó
1. `src/modules/entradas/api.ts`	• Se agregó la función `consolidarReservaYEmitirTicket` , que llama a `POST ${GATEWAY_URL}/api/entradas/reservas/{idReserva}/webhook-pago` pasando por el API Gateway (`:8080`) y devuelve el `id_entrada` real creado por el backend en MongoDB. • En `emitirEntradaDirecta` , se conectó al endpoint real `POST ${GATEWAY_URL}/api/entradas/emision-directa` para que las entradas gratuitas también se creen en MongoDB y se sincronicen con Check-in. • En `obtenerDetalleTicket` , se priorizó la consulta real al backend (`GET ${GATEWAY_URL}/api/entradas/{idEntrada}/detalle`) antes de buscar en sessionStorage, asegurando la visualización de datos reales (id_entrada, url_qr, estado_entrada, etc.).

Archivo en FrontendTitec-main	Qué se modificó
2. `src/modules/entradas/components/ModalReservaHold.tsx`	• **Corrección del bug de integración:** Anteriormente realizaba un `fetch` a `/api/entradas/reservas/...` sin `${GATEWAY_URL}`, impactando a Next.js en el puerto `:3000` en vez del Gateway en `:8080` (generando error 404) e inventando un ID aleatorio. • Ahora, `handlePagar` invoca `consolidarReservaYEmitirTicket(reserva.idReserva)`, recibe el `idTicketEmitido` real de MongoDB y redirige a la ruta correcta `/entradas/confirmacion/{idTicketReal}`.
3. `src/modules/entradas/components/EntradasPlaceholder.tsx`	• Se reemplazó la versión antigua estática (3 tarjetas) por el componente interactivo completo. • Ahora consulta `GET /api/entradas/eventos` y `GET /api/entradas/eventos/{id}/disponibilidad`. • Muestra cupos disponibles en tiempo real, permite alternar entre eventos gratuitos y pagados, e inicia el flujo real de reserva/emisión.
4. `src/modules/entradas/components/ConfirmacionTicket.tsx`	• Se actualizó con la vista completa que incluye el botón para copiar el `idEntrada` al portapapeles (facilitando la prueba en `/checkin`). • Se habilitó la descarga del comprobante en formato `.txt` y la visualización del código QR real.
5. `src/app/entradas/page.tsx`	• Se sincronizó el encabezado estructural de la página con la versión final del microservicio.

Archivo en FrontendTitec-main	Qué se modificó
6. `tsconfig.json` y `next.config.js`	• El repositorio contenía una subcarpeta `frontend/` duplicada e incompleta de otro grupo que bloqueaba la compilación en Docker (`npm run build`) por errores en `frontend/src/app/admin/login/page.tsx` . • Se excluyó el directorio `"frontend"` en `tsconfig.json` . • Se habilitó `ignoreBuildErrors` en `next.config.js` para permitir que el contenedor de Docker compile exitosamente sin ser bloqueado por módulos ajenos.